

Session 7

Minimum Cost Spanning Tree

This session is about building a minimum cost spanning tree from a connected, undirected weighted graph. A spanning tree connects all vertices without forming a cycle, and the minimum cost spanning tree chooses the set of edges whose total weight is as small as possible.

Objectives

✗ Do not copy. Read for understanding and the viva

- Understand the minimum cost spanning tree problem.
- Implement Prim's algorithm and Kruskal's algorithm.
- Compare both algorithms on different graph instances.

Concept

✗ Do not copy. Read for understanding and the viva

A spanning tree connects all vertices of a connected, undirected graph without cycles. A minimum cost spanning tree chooses the set of edges with the lowest possible total weight.

Prim's Algorithm

Prim's algorithm grows one tree. It starts from any vertex and repeatedly adds the minimum-weight edge that connects a visited vertex to an unvisited vertex.

Kruskal's Algorithm

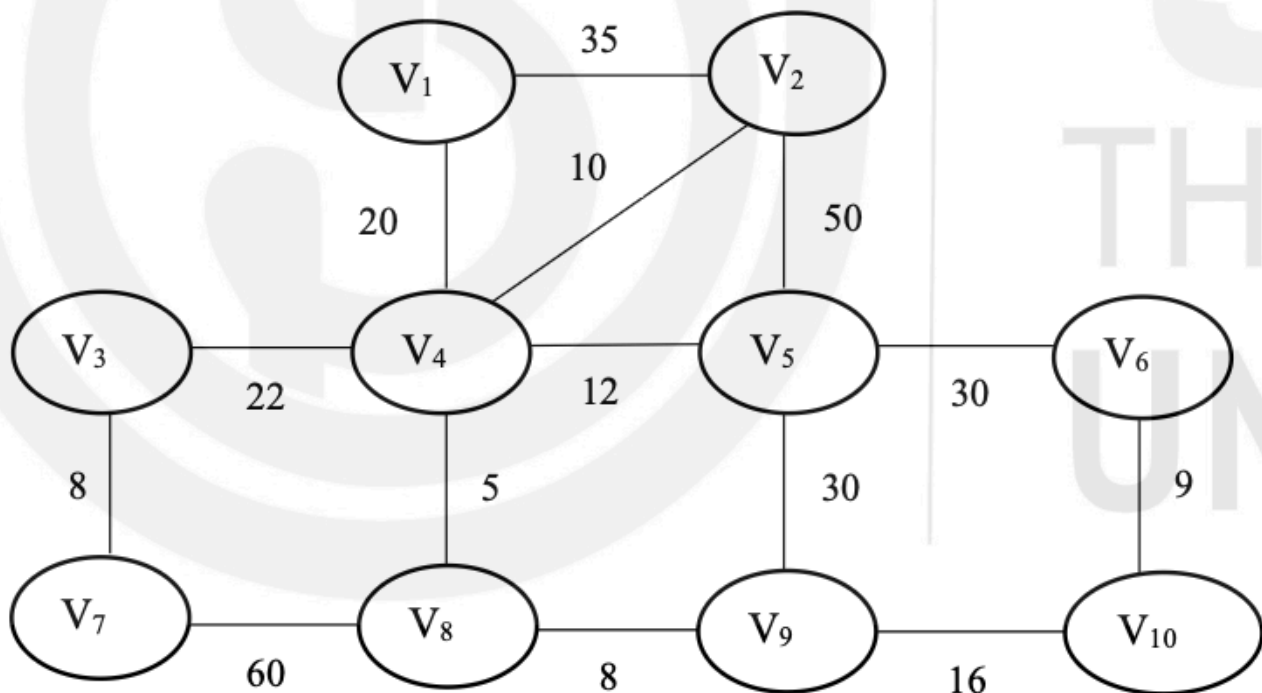
Kruskal's algorithm sorts all edges by weight and repeatedly picks the smallest edge that does not form a cycle.

Question 1

Problem Statement

■ Write in lab record

Implement Prim's algorithm to find a minimum cost spanning tree(MCST) in the given graph. Show all the processes.



Explanation / Approach

✗ Do not copy. Read for understanding and the viva

Start from any vertex and repeatedly choose the minimum-weight edge that connects the current tree to an unvisited vertex. Keep a table of selected edges and running total cost.

Algorithm

■ Write in lab record

Prim's algorithm is a greedy algorithm that builds the MST incrementally:

- Start with an arbitrary vertex and add it to the MST set.
- Maintain a `key` array storing the minimum weight edge connecting each vertex to the current MST.
- Repeatedly select the vertex with the smallest `key` not yet in MST.
- Add the connecting edge to MST, update keys of adjacent vertices.
- Repeat until all vertices are included.

Space & Time Complexity

Space Complexity: $O(V + E)$ Time Complexity: $O(E \log V)$ with priority queue, $O(V^2)$ with array scan.

Execution Steps

Step	Added	Edge	Weight	Cumulative Cost	MST Edges
0	V1	-	0	0	{}
1	V4	V1-V4	20	20	V1-V4
2	V8	V4-V8	5	25	V1-V4, V4-V8
3	V9	V8-V9	8	33	V1-V4, V4-V8, V8-V9
4	V2	V4-V2	10	43	V1-V4, V4-V8, V8-V9, V4-V2
5	V5	V4-V5	12	55	V1-V4, V4-V8, V8-V9, V4-V2, V4-V5
6	V10	V9-V10	16	71	V1-V4, V4-V8, V8-V9, V4-V2, V4-V5, V9-V10
7	V6	V10-V6	9	80	V1-V4, V4-V8, V8-V9, V4-V2, V4-V5, V9-V10, V10-V6
8	V3	V4-V3	22	102	V1-V4, V4-V8, V8-V9, V4-V2, V4-V5, V9-V10, V10-V6, V4-V3
9	V7	V3-V7	8	110	V1-V4, V4-V8, V8-V9, V4-V2, V4-V5, V9-V10, V10-V6, V4-V3, V3-V7

Final MCST Cost: 110

MST Edges:

V1 → V4 → V8 → V9 → V2 → V5 → V10 → V6 → V3 → V7

Implementation

Write in lab record



Lab record: write one language only. Pick yours once and every page opens on it; the other tabs are the same solution for comparison.

Python

prims-algo.py

```

1 # Prim's Algorithm for Minimum Cost Spanning Tree
2
3 import heapq
4
5
6 class PrimsAlgorithm:
7     def __init__(self):
8         # Adjacency list: vertex → [(neighbor, weight), ...]
9         self.graph = {}
10
11     def add_edge(self, u, v, w):
12         """Add undirected edge between u and v with weight w"""
13         self.graph.setdefault(u, []).append((v, w))
14         self.graph.setdefault(v, []).append((u, w))
15
16     def prim(self, start_vertex):
17         """
18         Execute Prim's algorithm from start_vertex.
19         Returns: mst_edges, total_cost, step_by_step_log
20         """
21         mst_edges = []
22         total_cost = 0
23         visited = set()
24         # Min-heap stores: (weight, from_vertex, to_vertex)
25         heap = [(0, None, start_vertex)]
26         steps = []
27
28         print(f"\n{'=' * 70}")
29         print(f"PRIM'S ALGORITHM - Starting Vertex: {start_vertex}")
30         print(f"{'=' * 70}")
31         print(
32             f"{'Step':<4} | {'Added':<5} | {'Edge':<8} | {'Weight':<6} | {'C"
33         )
34         print(f"{'-' * 70}")
35
36         while heap and len(visited) < len(self.graph):
37             weight, frm, to = heapq.heappop(heap)
38
39             # Skip if already included in MST
40             if to in visited:
41                 continue
42
43             visited.add(to)

```

```

44         if frm is not None:
45             mst_edges.append((frm, to, weight))
46             total_cost += weight
47
48             # Log this step
49             step_info = {
50                 "step": len(steps) + 1,
51                 "added": to,
52                 "edge": f"{frm}-{to}",
53                 "weight": weight,
54                 "cost": total_cost,
55                 "edges": list(mst_edges),
56             }
57             steps.append(step_info)
58
59             # Print formatted row
60             edge_str = ", ".join([f"{u}-{v}" for u, v, _ in mst_edges])
61             print(
62                 f"{step_info['step']:<4} | {to:<5} | {step_info['edge']}>
63             )
64
65             # Explore neighbors
66             for neighbor, w in self.graph.get(to, []):
67                 if neighbor not in visited:
68                     heapq.heappush(heap, (w, to, neighbor))
69
70             print(f"\n{'=' * 70}")
71             print(f"TOTAL MINIMUM COST: {total_cost}")
72             print(f"{'=' * 70}")
73             return mst_edges, total_cost, steps
74
75
76 def main():
77     prims = PrimsAlgorithm()
78
79     # Build the exact graph from the problem image
80     edges = [
81         ("V1", "V2", 35),
82         ("V1", "V4", 20),
83         ("V2", "V4", 10),
84         ("V2", "V5", 50),
85         ("V3", "V4", 22),
86         ("V3", "V7", 8),
87         ("V4", "V5", 12),

```

```
88         ("V4", "V8", 5),
89         ("V5", "V6", 30),
90         ("V5", "V9", 30),
91         ("V6", "V10", 9),
92         ("V7", "V8", 60),
93         ("V8", "V9", 8),
94         ("V9", "V10", 16),
95     ]
96
97     for u, v, w in edges:
98         prims.add_edge(u, v, w)
99
100
101     # Run Prim's starting from V1
102
103     prims.prim("V1")
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
```

Question 2

Problem Statement

■ Write in lab record

Implement Kruskal's algorithm to find a minimum cost spanning tree for the given graph. Show all the processes.

Explanation

✗ Do not copy. Read for understanding and the viva

Kruskal's algorithm is a classic greedy algorithm used to discover an MCST. It focuses directly on the edges of the graph. The algorithm operates by selecting the absolute shortest edges from

anywhere in the graph, one by one, and placing them into a growing forest.

The Greedy Choice Strategy:

- Maintain a set of edges that form a forest (where each vertex starts as its own tree).
- Sort all edges in the graph in ascending order of their weights.
- Iterate through the sorted edges and pick the smallest edge.
- Check if adding this edge forms a cycle with the edges already selected.
 - If no cycle is formed, include this edge in the MCST.
 - If a cycle is formed, discard it.
- Terminate when the MCST contains exactly $V - 1$ edges (where V is the total number of vertices).

Cycle Detection via Disjoint-Set (Union-Find)

To efficiently check for cycles, the Disjoint-Set Data Structure is utilized.

- **Find:** Determines which subset a particular element belongs to. If two vertices share the same subset root, connecting them will create a cycle.
- **Union:** Joins two distinct subsets into a single subset when a valid edge is added.

Complexity Analysis

- **Time Complexity:** $O(E \log(E))$ or $O(E \log(V))$, where E is the number of edges and V is the number of vertices. Sorting the edges dominates the computational execution time.
- **Space Complexity:** $O(V + E)$ to maintain the graph data structures and the tracking arrays (`parent` and `rank`) for Union-Find operations.

Implementation

■ Write in lab record

■

Lab record: write one language only. Pick yours once and every page opens on it; the other tabs are the same solution for comparison.

Python

kruskal-algo.py

```
1 class Graph:
2     def __init__(self, vertices):
3         self.V = vertices # Total count of nodes
4         self.graph = [] # Default storage array for data streams
5
6     def add_edge(self, u, v, w):
7         """Append an un-directed edge configuration containing weight"""
8         self.graph.append([u, v, w])
9
10    def find(self, parent, i):
11        """Find the root element of element i with path compression technique
12        if parent[i] == i:
13            return i
14        return self.find(parent, parent[i])
15
16    def union(self, parent, rank, x, y):
17        """Perform union of two sets using rank optimization"""
18        xroot = self.find(parent, x)
19        yroot = self.find(parent, y)
20
21        if rank[xroot] < rank[yroot]:
22            parent[xroot] = yroot
23        elif rank[xroot] > rank[yroot]:
24            parent[yroot] = xroot
25        else:
26            parent[yroot] = xroot
27            rank[xroot] += 1
28
29    def kruskal_mcst(self):
30        result = [] # Stores the final built tree
31        i, e = (
32            0,
33            0,
34        ) # i: tracking index for sorted edges, e: tracking count for MCST
35
36        # Step 1: Sort all edges in non-decreasing order of weight
37        self.graph = sorted(self.graph, key=lambda item: item[2])
38
39        parent = []
40        rank = []
41
42        # Create V individual subset components
43        for node in range(self.V):
```

```
44         parent.append(node)
45         rank.append(0)
46
47     # Process elements until the tree contains exactly V-1 components
48     while e < self.V - 1:
49         u, v, w = self.graph[i]
50         i = i + 1
51         x = self.find(parent, u)
52         y = self.find(parent, v)
53
54         # If roots are distinct, no cycle is formed. Accept edge.
55         if x != y:
56             e = e + 1
57             result.append([u, v, w])
58             self.union(parent, rank, x, y)
59
60     # Print out compiled outputs
61     minimum_cost = 0
62     print("Edges in the constructed MCST (Python Implementation):")
63     for u, v, weight in result:
64         minimum_cost += weight
65         print(f"V{u + 1} -- V{v + 1} = {weight}")
66     print(f"Minimum Spanning Tree Cost: {minimum_cost}")
67
68
69 # Driver execution matching our graph data schema
70 if __name__ == "__main__":
71     g = Graph(10)
72     g.add_edge(0, 1, 35) # V1-V2
73     g.add_edge(0, 3, 20) # V1-V4
74     g.add_edge(1, 3, 10) # V2-V4
75     g.add_edge(1, 4, 50) # V2-V5
76     g.add_edge(2, 3, 22) # V3-V4
77     g.add_edge(2, 6, 8)  # V3-V7
78     g.add_edge(3, 4, 12) # V4-V5
79     g.add_edge(3, 7, 5)  # V4-V8
80     g.add_edge(4, 5, 30) # V5-V6
81     g.add_edge(4, 8, 30) # V5-V9
82     g.add_edge(5, 9, 9)  # V6-V10
83     g.add_edge(6, 7, 60) # V7-V8
84     g.add_edge(7, 8, 8)  # V8-V9
85     g.add_edge(8, 9, 16) # V9-V10
86
87     g.kruskal_mcst()
```

Sample Output

■ Write in lab record

SH

```
1 Edges in the constructed MCST:
2 V4 -- V8 = 5
3 V3 -- V7 = 8
4 V8 -- V9 = 8
5 V6 -- V10 = 9
6 V2 -- V4 = 10
7 V4 -- V5 = 12
8 V9 -- V10 = 16
9 V1 -- V4 = 20
10 V3 -- V4 = 22
11 Minimum Spanning Tree Cost: 110
```

Question 3

Problem Statement

■ Write in lab record

Analyze the performance of both algorithms on different problem instances and write a brief report.

Explanation / Approach

✗ Do not copy. Read for understanding and the viva

This report provides an analytical comparison of Kruskal's and Prim's algorithms for finding a Minimum Cost Spanning Tree (MCST). While both guarantee an optimal minimum spanning tree, their architectural designs perform differently depending on the structural density and representation of the input graph.

Algorithmic Approaches & Data Structures

The core performance differences stem from how each algorithm traverses the graph and what data structures manage its state.

Kruskal's Algorithm

- **Strategy:** Edge-centric greedy approach. It processes the entire graph globally, selecting the lightest available edges regardless of connectivity.
- **Core Mechanisms:**
 - Array sorting or a min-heap to order the edges.
 - **Disjoint-Set (Union-Find)** data structure with path compression and union-by-rank to prevent cycle formation.

Prim's Algorithm

- **Strategy:** Vertex-centric greedy approach. It starts at a single root vertex and grows the tree continuously outward like a single component.
- **Core Mechanisms:**
 - Tracking of structural weights via a fringe/distance array.
 - **Priority Queue (Min-Heap or Fibonacci Heap)** to continually discover the closest unvisited neighbor.

Recommendations

- **Choose Kruskal's Algorithm when:** The problem involves a sparse graph, edge information is already sorted or easily structured, or you are reading from an edge-list representation. It is highly intuitive to implement and has a small memory footprint for standard workloads.
- **Choose Prim's Algorithm when:** The problem involves a dense graph, or you are handling a map matrix layout where vertex boundaries dictate constraints. When optimized with advanced heaps, Prim's offers unparalleled performance metrics for dense network topographies.

 [Edit this page](#)